Simulation 7: Dynamic SQL Execution and Security (DSL)

Author: Suhani Mehta
N01750525

1. Purpose of Simulation

The purpose of this simulation is to understand and implement dynamic SQL execution in SQL Server while exploring the differences between secure and vulnerable approaches.

This simulation :

- How to build parameterized dynamic SQL safely
- How to identify unsafe SQL practices
- How to implement logging and input validation
- How to analyze procedure execution and security behavior.

2. Description of Secure and Vulnerable Approach

Secure Procedure (usp_SecureSalesReport)

- Uses sp_executesql to execute dynamic SQL safely.
- Supports optional filters: Territory, Salesperson, Product Category, Start Date, End Date.
- Implements input validation to reject unsafe input patterns, such as --, DROP, and EXEC.
- Logs all execution attempts in the ExecutionLog table, including procedure name, parameters, execution status, and timestamp.
- Returns meaningful results without exposing the system to SQL injection attacks.

Example execution:

```
EXEC Reporting.usp_SecureSalesReport
   @TerritoryName='Northwest',
   @StartDate='2023-01-01',
   @EndDate='2023-12-31';
```

Vulnerable Procedure (usp_VulnerableSalesReport)

- Constructs dynamic SQL using string concatenation, inserting parameter values directly into the query.
- Supports the same optional filters.
- Susceptible to SQL injection, allowing attackers to manipulate query logic or execute unintended commands.
- Logs all executions, but unsafe input can still compromise results.

Example vulnerable input:

```
EXEC Reporting.usp_VulnerableSalesReport
   @TerritoryName='Northwest"; DROP TABLE Sales.SalesOrderHeader; --';
```

3. Observations from Testing

During testing, the following behaviors were observed:

1. Secure Procedure Testing

- Executes successfully with valid input.
- Unsafe input (e.g., containing `DROP`) is rejected, and the attempt is recorded as Rejected in the ExecutionLog.
- Optional parameters worked correctly, returning filtered results when provided or all results when parameters were NULL.

2. Vulnerable Procedure Testing

- Executes normally with valid input.
- Malicious input can alter query logic, demonstrating SQL injection risk.
- ExecutionLog captured the run, but the procedure could be exploited in a real environment if unsafe input is used.

3. Execution Logging and Summary

- ExecutionLog clearly shows procedure name, parameter values, execution status (Success, Failed, Rejected), and timestamp.
- Summary view aggregates execution counts, providing insight into the number of successful, failed, and rejected executions.

4. Explanation of SQL Injection

SQL injection occurs when user input is treated as executable code rather than as data.

In dynamic SQL built via string concatenation (vulnerable procedure), malicious input can:

- Alter WHERE clauses to return unauthorized data
- Inject destructive commands like DROP or DELETE
- Compromise the integrity of the database

Example of SQL injection input:

```
@TerritoryName='Northwest"; DROP TABLE Sales.SalesOrderHeader; --'
```

- This input closes the existing query string and executes a DROP TABLE command.
- Parameterized queries in the secure procedure prevent this because user input is treated as data, not SQL code.

5. Key Conclusions

1. Parameterized dynamic SQL is essential for preventing SQL injection.
2. Input validation and logging enhance system security and auditability.
3. Vulnerable dynamic SQL exposes systems to serious risks and should never be used in production.
4. Optional filtering parameters improve query flexibility but must be implemented securely.
5. Execution logs and summary views provide visibility into procedure usage and potential abuse.